PROJETO INTERDISCIPLINAR

ACERTE O ALVO

ENTREGA 02 — IMPLEMENTAÇÃO FINAL DO SISTEMA

Relatório técnico de arquitetura, integração, funcionamento e análise crítica

Resumo

O ACERTE O ALVO é um sistema interativo composto por uma estrutura física com cinco alvos numerados, cada um equipado com um sensor piezoelétrico, e uma aplicação digital acessível por desktop e dispositivos móveis. Na configuração final descrita pela equipe, a aplicação apresenta o alvo que deve ser atingido, controla o tempo da tentativa e administra a pontuação. O Arduino realiza a leitura dos sensores e se comunica com a aplicação por meio de cabo USB conectado ao notebook. A estrutura física funciona como módulo de entrada de impactos, enquanto a aplicação concentra a interface e a lógica do jogo. Funcionalidades inicialmente previstas para o circuito, como LEDs, buzzer e contador físico, foram retiradas ou transferidas para a aplicação.

1. Introdução

A segunda entrega apresenta a implementação efetivamente desenvolvida, evidenciando a integração entre os componentes físicos e digitais e analisando a solução final em relação à proposta inicial.

O objetivo do ACERTE O ALVO é oferecer uma experiência de precisão na qual o participante deve atingir, em uma estrutura física, o alvo indicado pela aplicação. A interface informa o alvo solicitado, o tempo disponível e a pontuação do jogador. O impacto é captado pelo sensor piezoelétrico correspondente, lido pelo Arduino e encaminhado ao software por conexão USB.

A solução divide-se em dois módulos: (i) módulo físico de detecção, formado pelos cinco alvos e seus sensores piezoelétricos conectados ao Arduino; e (ii) aplicação digital, responsável pela interação com o jogador e pelo gerenciamento da partida.

2. Arquitetura final implementada

2.1. Componentes e responsabilidades

Componente	Função na solução final
Estrutura física com cinco alvos	Permitir impactos em alvos numerados de 1 a 5.
Cinco sensores piezoelétricos	Detectar impactos individualmente em cada alvo.
Arduino	Ler os sinais dos sensores e transmitir à aplicação a identificação do alvo atingido.
Cabo USB e notebook	Estabelecer a comunicação entre Arduino e aplicação executada no notebook.
Aplicação digital	Indicar o alvo solicitado, controlar o tempo, tratar o evento recebido e administrar a pontuação.
Interface desktop e móvel	Permitir o acompanhamento da partida em computadores e dispositivos móveis, conforme informado pela equipe.

2.2. Fluxo arquitetural

Camada	Elementos	Responsabilidade
Entrada física	Alvos 1 a 5 e piezoelétricos	Capturar o impacto do jogador.
Aquisição	Arduino	Realizar a leitura dos cinco canais analógicos.
Comunicação	USB / serial	Encaminhar ao software a informação do impacto.
Processamento do jogo	Aplicação digital	Indicar alvo, controlar tempo, validar resultado e atualizar pontuação.
Saída ao usuário	Tela da aplicação	Exibir alvo solicitado, tempo e pontuação.

A arquitetura final não utiliza LEDs, buzzer ou contador físico. A indicação do alvo e as informações da partida são apresentadas na tela da aplicação.

2.3. Alterações em relação à proposta inicial

Durante o desenvolvimento, a equipe adaptou a distribuição das funcionalidades. A proposta inicial previa sinalização e controle no circuito físico. Na versão final, foram mantidos os cinco alvos e os cinco sensores piezoelétricos, mas os

LEDs, o buzzer e a contagem de tempo no hardware não foram implementados. O controle do tempo, a indicação do alvo e a pontuação passaram a ser responsabilidades da aplicação digital.

Elemento previsto	Situação final	Descrição
Cinco alvos físicos	Implementado	Mantidos e numerados de 1 a 5.
Cinco piezoelétricos	Implementado	Um sensor por alvo.
Arduino	Implementado	Leitura dos sensores e comunicação USB.
LEDs físicos	Não implementado	Não fazem parte da estrutura final.
Buzzer	Não implementado	Não há sinalização sonora física.
Contagem regressiva no circuito	Transferida ao software	Tempo controlado pela aplicação.
Indicação do alvo e pontuação	Na aplicação	Interface apresenta o alvo e a pontuação.

3. Implementação do hardware

A estrutura física é composta por uma placa com cinco alvos identificados numericamente. Cada alvo possui um sensor piezoelétrico associado. Quando ocorre um impacto, o piezoelétrico gera uma variação elétrica que pode ser lida pelo Arduino por uma entrada analógica. A associação individual entre cada sensor e sua posição permite identificar qual alvo recebeu o impacto.

No arquivo codigo_teste_piezo, os sensores são associados às entradas A0, A1, A2, A3 e A4. O programa realiza leituras analógicas sucessivas, calcula a diferença entre a leitura atual e a anterior e considera um evento quando a variação supera DELTA_MINIMO, definido como 20, respeitando também um intervalo de debounce de 120 milissegundos por sensor.

O código de teste envia pela porta serial uma mensagem textual com o número do alvo, a variação detectada e a leitura absoluta. Essa rotina demonstra uma estratégia de detecção por variação do sinal (delta), distinta da estratégia por limiar absoluto encontrada no segundo firmware.

A montagem física final informada pela equipe não possui LEDs, buzzer ou contador. Embora o arquivo smart_tennis_arduino contenha rotinas para esses dispositivos, sua presença no código não significa que estejam instalados ou sejam utilizados na versão física final.

4. Implementação do software e comunicação

4.1. Comunicação Arduino–aplicação

A comunicação física entre Arduino e notebook é realizada por cabo USB. Nos códigos fornecidos, a comunicação serial é inicializada a 115200 bps por meio de Serial.begin(115200). O Arduino transmite mensagens pela porta serial, que podem ser recebidas pela aplicação conectada à porta correspondente.

No codigo_teste_piezo, a mensagem transmitida é textual e contém a identificação do alvo, a variação (delta) e a leitura absoluta. No arquivo smart_tennis_arduino, os eventos são estruturados em formato semelhante a JSON, com campos como evt, tgt, val e ts. A aplicação precisa interpretar o formato efetivamente enviado pelo firmware carregado na placa.

4.2. Responsabilidades da aplicação

Conforme a descrição funcional da equipe, a aplicação funciona em desktop e dispositivos móveis e concentra a dinâmica da partida. Ela apresenta qual alvo deve ser atingido, informa o tempo disponível e exibe a pontuação. Ao receber a informação do impacto, o software identifica o alvo correspondente e trata o resultado conforme as regras do jogo.

O código-fonte da aplicação digital não foi incluído entre os arquivos disponibilizados para esta versão do relatório. Por isso, a descrição de sua interface e de suas funcionalidades está baseada na explicação fornecida pela equipe, e não em uma análise direta do código da aplicação.

5. Análise dos códigos-fonte disponibilizados

5.1. codigo_teste_piezo

O primeiro arquivo é um programa de teste dos cinco sensores piezoelétricos. Define as entradas analógicas A0 a A4, utiliza DELTA_MINIMO igual a 20 e aplica debounce de 120 ms. Em cada ciclo, lê o sensor, calcula a diferença entre a leitura atual e a anterior e envia uma mensagem serial quando identifica uma variação acima do limite e respeita o intervalo mínimo entre detecções.

Parâmetro	Configuração observada
Entradas dos sensores	A0, A1, A2, A3 e A4.
Critério de detecção	Variação positiva superior a DELTA_MINIMO.
DELTA_MINIMO	20 unidades de leitura ADC.
Debounce	120 milissegundos por sensor.
Comunicação serial	115200 bps.
Mensagem transmitida	Número do alvo, delta e leitura absoluta.

5.2. smart_tennis_arduino

O segundo arquivo contém um firmware mais abrangente, identificado no cabeçalho como Smart Tennis Arena para Arduino Uno R3 (ATmega328P). O programa define cinco sensores piezoelétricos nas entradas A0 a A4 e limiares iniciais de 100 para cada sensor. Também implementa uma máquina de estados com inicialização, espera, alvo ativo, pausa e parada de emergência.

O firmware possui comunicação serial a 115200 bps, leitura dos sensores, comandos START, PAUSE, RESUME, RESET, PING, FEED, BUZZ, SET_LEDS, CALIB e POLL, além de eventos como READY, HIT, WRONG_HIT, MISS e ESTOP_ALERT. A lógica de detecção durante uma rodada prioriza o alvo ativo, utiliza lockout de 180 ms e verifica os demais sensores para identificar impacto em alvo diferente.

Entretanto, esse arquivo também contém rotinas e definições para LEDs, buzzer, botão de emergência e máquina de bolas. Como a equipe informou que esses componentes não integram a montagem física final, tais recursos são considerados funcionalidades presentes no código fornecido, mas não implementadas na estrutura física final descrita neste relatório.

Recurso no firmware	O que o código apresenta	Situação física informada
Sensores piezoelétricos	Cinco entradas analógicas e leitura por limiar.	Presentes na montagem.
Comunicação serial	Mensagens de eventos e comandos em formato JSON.	USB conectado ao notebook.
LEDs	Pinos 3 a 7 e funções de acionamento.	Não instalados na versão final.
Buzzer	Pino 8 e funções sonoras.	Não instalado na versão final.
Botão de emergência	Pino 2 com interrupção externa.	Não informado como parte da montagem final.
Máquina de estados	BOOT, IDLE_READY, TARGET_ACTIVE, PAUSED e EMERGENCY_STOP.	Lógica disponível no firmware; uso efetivo depende da versão carregada.
Comandos seriais	START, PAUSE, RESUME, RESET, PING, entre outros.	Dependem do protocolo utilizado pela aplicação.

5.3. Compatibilidade entre firmware e aplicação

Os dois arquivos fornecidos utilizam formatos de saída serial diferentes. O codigo_teste_piezo transmite mensagens textuais de diagnóstico, enquanto smart_tennis_arduino transmite eventos estruturados em formato JSON. Para que a aplicação reconheça corretamente os impactos, o firmware carregado no Arduino e o leitor serial da aplicação precisam utilizar o mesmo formato de mensagem.

Assim, a documentação distingue o programa de teste dos sensores do firmware Smart Tennis mais completo. A confirmação de qual arquivo foi efetivamente gravado no Arduino durante a demonstração é necessária para associar o comportamento observado ao código correspondente. O relatório não presume que os dois programas tenham sido executados simultaneamente.

6. Demonstração do fluxo completo de dados e ações

1. A aplicação inicia a partida e apresenta ao jogador o alvo solicitado.
2. O jogador identifica na estrutura física o número correspondente ao alvo indicado na tela.
3. O jogador atinge o alvo e o piezoelétrico correspondente gera uma variação elétrica.
4. O Arduino realiza a leitura do sensor conectado à entrada analógica correspondente.
5. O Arduino envia pela conexão USB/serial a informação referente ao impacto detectado.
6. A aplicação recebe o evento e identifica qual alvo foi atingido.
7. A aplicação trata o resultado conforme as regras do jogo e atualiza pontuação, tempo e indicação da próxima ação.

Esse fluxo representa a operação funcional descrita pela equipe. O detalhamento do protocolo e das regras internas de pontuação depende da implementação da aplicação, cujo código não foi disponibilizado.

7. Testes realizados e validação dos requisitos

Os arquivos fornecidos permitem analisar a lógica de leitura dos sensores e os mecanismos previstos para comunicação serial. O codigo_teste_piezo apresenta detecção por variação de sinal e envio de mensagens de diagnóstico; o firmware Smart Tennis apresenta detecção por limiar, debounce, estados operacionais e emissão de eventos seriais. Os códigos, isoladamente, não comprovam resultados de testes físicos nem fornecem medições de taxa de acerto, latência ou confiabilidade.

Requisito	Critério de verificação	Base disponível
Detecção dos cinco alvos	Verificar se cada sensor é identificado individualmente.	Código de teste associa A0–A4 aos alvos 1–5.
Identificação do impacto	Verificar se a mensagem serial indica o alvo.	Código de teste imprime o alvo; Smart Tennis envia campo tgt.
Comunicação USB/serial	Verificar recepção das mensagens no notebook/aplicação.	Ambos os códigos inicializam Serial a 115200 bps.
Indicação do alvo solicitado	Verificar se a aplicação apresenta o alvo a atingir.	Funcionalidade descrita pela equipe.
Controle de tempo	Verificar se a aplicação controla o tempo da tentativa.	Funcionalidade descrita; não demonstrada pelos códigos Arduino enviados.
Pontuação	Verificar atualização dos pontos após a jogada.	Funcionalidade descrita; lógica não disponível nos arquivos enviados.
Compatibilidade da interface	Verificar operação em desktop e dispositivo móvel.	Compatibilidade informada pela equipe; código da aplicação não enviado.

A análise documental confirma que os códigos incluem rotinas para leitura de cinco sensores e transmissão serial. A comprovação empírica de cada requisito depende da demonstração física e do registro dos testes realizados pela equipe. Não foram atribuídos resultados de aprovação sem evidência documental.

8. Código-fonte, organização e versionamento

Foram disponibilizados dois arquivos de código relacionados ao Arduino: codigo_teste_piezo, voltado ao teste dos cinco sensores por variação do sinal, e smart_tennis_arduino, que contém um firmware mais amplo com máquina de estados, comandos seriais e eventos estruturados.

Os arquivos demonstram a existência de versões ou abordagens distintas para o firmware. Não foi fornecido endereço de repositório, histórico de commits ou código-fonte da aplicação digital. Portanto, não é possível afirmar, com base no material recebido, que o projeto completo esteja versionado em uma plataforma de controle de versão. Esta documentação registra os arquivos-fonte disponibilizados para análise.

9. Backlog final atualizado

O backlog a seguir consolida as funcionalidades descritas para a versão final e as alterações identificadas em relação à proposta inicial.

Item	Situação final	Descrição
Construção da estrutura com cinco alvos	Implementado	Estrutura com alvos numerados de 1 a 5.
Instalação de cinco piezoelétricos	Implementado	Um sensor associado a cada alvo.
Leitura dos sensores pelo Arduino	Implementado no firmware fornecido	Ambos os arquivos contêm leitura dos cinco sensores.
Comunicação Arduino–aplicação por USB	Implementado conforme descrição	Conexão via cabo USB ao notebook.
Aplicação para desktop e dispositivos móveis	Implementado conforme descrição	Interface concentra a dinâmica do jogo.
Indicação do alvo na aplicação	Implementado conforme descrição	A tela informa qual alvo deve ser atingido.
Controle de tempo pela aplicação	Implementado conforme descrição	Tempo administrado digitalmente.
Pontuação pela aplicação	Implementado conforme descrição	Pontuação administrada digitalmente.
LEDs físicos	Não implementado	Recurso retirado da versão final.
Buzzer físico	Não implementado	Recurso retirado da versão final.
Contador físico	Não implementado	Controle de tempo transferido para a aplicação.
Versionamento do projeto completo	Não comprovado pelo material recebido	Não foi fornecido repositório ou histórico de versões.
Código-fonte da aplicação	Não disponibilizado para esta análise	A descrição funcional foi baseada no relato da equipe.

10. Limitações e propostas de melhoria

10.1. Limitações identificadas

- Dependência de conexão física USB entre o Arduino e o notebook para transmitir os eventos dos sensores.
- A estrutura física não possui feedback luminoso ou sonoro, pois LEDs e buzzer não foram implementados.
- A experiência do jogador depende da aplicação para visualizar o alvo solicitado, o tempo e a pontuação.
- Os sensores piezoelétricos podem exigir calibração de sensibilidade e tratamento de ruídos e vibrações para reduzir detecções indevidas.
- Os dois códigos fornecidos utilizam formatos de mensagem serial diferentes, o que exige alinhamento entre o firmware carregado e o protocolo interpretado pela aplicação.
- O código-fonte da aplicação e evidências quantitativas dos testes não fizeram parte dos materiais disponibilizados para esta documentação.

10.2. Propostas de melhoria

- Adicionar LEDs aos alvos para oferecer indicação visual imediata.
- Adicionar sinalização sonora para acertos, erros e encerramento de rodada.
- Avaliar comunicação sem fio como alternativa ao cabo USB, mantendo a confiabilidade da transmissão.
- Implementar rotina de calibração dos piezoelétricos e filtros para aumentar a precisão da detecção.
- Padronizar o protocolo serial entre Arduino e aplicação, adotando mensagens estruturadas e tratamento de falhas de comunicação.
- Disponibilizar o código-fonte da aplicação e organizar o projeto completo em repositório versionado.
- Implementar histórico de partidas e armazenamento de pontuações em uma evolução futura.
- Executar testes sistemáticos de compatibilidade e usabilidade em diferentes dispositivos e tamanhos de tela.

11. Considerações finais

A implementação final do ACERTE O ALVO consolidou uma solução integrada entre uma estrutura física com cinco alvos instrumentados por sensores piezoelétricos e uma aplicação digital responsável pela dinâmica do jogo. O Arduino realiza a leitura dos sensores e a comunicação com a aplicação por meio de cabo USB conectado ao notebook.

Em relação à proposta inicial, houve uma redistribuição das funcionalidades: LEDs, buzzer e contador físico não foram implementados, enquanto a indicação do alvo, o controle de tempo e a pontuação foram concentrados na aplicação. Essa alteração definiu uma arquitetura na qual o hardware atua como módulo de entrada e o software assume o gerenciamento da experiência do jogador.

A análise dos códigos identificou duas abordagens de firmware: um programa de teste baseado na variação do sinal dos sensores e um firmware mais abrangente, com limiares, máquina de estados e protocolo serial estruturado. A consistência entre o firmware efetivamente utilizado e o protocolo interpretado pela aplicação é essencial para a integração.

A solução apresenta uma arquitetura integrada entre sensores, microcontrolador e aplicação digital, com oportunidades de evolução relacionadas à padronização da comunicação, à calibração dos sensores, ao feedback físico e à documentação completa do software.

Referências técnicas do projeto

Código-fonte codigo_teste_piezo.txt — programa de teste para leitura dos cinco sensores piezoelétricos, com detecção por variação (delta) e saída serial.

Código-fonte smart_tennis_arduino.txt — firmware Smart Tennis Arena para Arduino Uno R3, com leitura de sensores, máquina de estados, comandos seriais e emissão de eventos.

Descrição da implementação final e do funcionamento da aplicação — informações fornecidas pela equipe do projeto.